Project guide

Football Transfer Market

How the project was planned, built, verified and deployed: the full stack explained for learning, with diagrams, algorithms, problems hit and how they were avoided.

Live site: https://frontend-theta-three-tj1p5tsbe7.vercel.app

Repository: github.com/ssupreethsg/football-transfer-market (branch `claude/data-pipeline-v2`)

Scope: 2,707 players · 96 clubs · Premier League, La Liga, Serie A, Bundesliga, Ligue 1 · season 2026/27

Written: 31 October 2026

Contents

0. Read this first: the basic ideas in plain words
1. The project in one page
2. The starting plan: requirements, constraints and the framework we chose
3. Architecture: the big picture diagram
4. Technology stack: every language and tool, and why
5. Where we started: what was wrong with the first version
6. Data sources: what we used, what we rejected and why
7. The data pipeline, stage 1: collecting data (scraper)
8. The data pipeline, stage 2: transforming data (build)
9. The data model: how a player file is structured
10. The data pipeline, stage 3: verification
11. The frontend: how the website works
12. Hosting and deployment: Git, Vercel, preview and production
13. End-to-end workflow: what happens when you open a player page
14. Problems we hit and how we solved them
15. Other folders in the repository (legacy parts)
16. How to run, update and extend it
17. Lessons for your next project: a planning template and checklists
18. Glossary
19. How and where to learn all this (resources, method, using AI)
20. Roadmap: how to build a project like this from zero

0 Read this first: the basic ideas in plain words

This guide assumes you are a complete beginner. Before the technical chapters, here is the whole project explained with an everyday comparison. Come back to this page whenever a later chapter feels confusing.

The restaurant comparison

- **The market where ingredients come from** = the **data sources** on the internet (FotMob, Understat).
- **The shopper who buys ingredients and stores them in the pantry** = the **scraper** (`scrape_fotmob.py`) and its **cache** folder.
- **The kitchen that cooks the ingredients into dishes** = the **build step** (`build_dataset.py`): it turns raw data into neat, ready-to-serve files.
- **The head chef who tastes every dish before it leaves** = the **verification step** (`verify_dataset.py`).
- **The plates arranged on the counter** = the **JSON files** in `public/data/` .
- **The waiter and the dining room** = the **website (frontend)**: it takes what you ask for and presents the right plate nicely.
- **The building, its address and its doors** = **Vercel hosting**: it keeps the restaurant open 24/7 at a public address (URL).
- **The recipe book with every change written down** = **Git and GitHub**.

In our restaurant, all the cooking happens *before* the customers arrive. When a visitor opens the site, the dish is already on the counter, so service is instant and nothing can burn while they wait.

0.1 The words you will see everywhere

Word	Plain meaning	In this project
Frontend	The part of an app you see and click, which runs in your browser.	The website: home page with player cards, player pages with tables.
Backend	The part that runs on a computer somewhere else and prepares or stores data.	Our "backend" is the offline data pipeline. There is *no* always-on server.
Data pipeline	A chain of programs that each do one job on data, in order.	Collect → transform → verify.
API	A web address that answers with data instead of a web page.	FotMob's `/api/data/playerData?id=737066` answers with Haaland's data.
JSON	A simple text format for data that every language understands.	All our data files. Example: `{"name": "Haaland", "goals": 27}` .
Database	A program that stores data and answers questions about it.	The old version used SQLite. We replaced it with plain JSON files because nobody writes data on our site.
Framework	A ready-made structure so you don't build everything from zero.	Next.js (for the website).
Library	A package of ready-made code you call from your program.	`requests` (Python downloads), React (UI building blocks).
Repository (repo)	A project folder whose full history is tracked by Git.	`football-transfer-market` on GitHub.
Deploy	Put your app on the internet.	Upload to Vercel, which gives the public URL.

Terminal / command line	A text window where you type commands instead of clicking.	Where we ran `python build_dataset.py`, `npm run build`, `git push`.

0.2 How to read each chapter

Every chapter that describes part of the build ends with a blue box, 🧰 **What we used here, how, and why only this**. It lists each tool or technique from that chapter: what it is, exactly how we used it, and why we chose it over the alternatives. If you are learning with another AI, copy a row from that box and ask it to "teach me this from scratch with a small exercise". Purple boxes like the one above explain an idea in everyday words.

1 The project in one page

What it is. A website that lists every player at every club in Europe's top five leagues and shows, for each player, real statistics for his whole career: totals per competition (league, domestic cups, European competitions, national team) and a season-by-season breakdown where every season is split into each competition he played.

How it works, in one sentence. A Python pipeline downloads real match data, turns it into clean JSON files, checks them against an independent source, and a Next.js (TypeScript/React) website hosted on Vercel reads those files and displays them.

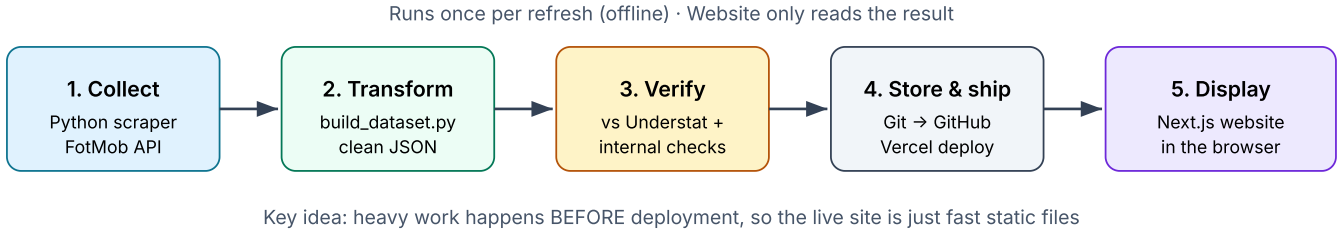


Figure 1. The five steps of the system.

Key numbers

Item	Value
Clubs / players	96 clubs, 2,707 players (current 2026/27 squads)
Requests made to collect the data	~28,400 (5 league tables, 96 squads, 2,707 profiles, 25,628 detailed-stat entries, 1 league list)
Detailed season-competition entries	23,693 (minutes, xG, dribbles, tackles… for 2023/24 onwards)
Dataset size shipped to the site	~57 MB in 2,709 JSON files (1 index, 1 meta, 2,707 player files)
Independent verification	League apps/goals/assists agree with Understat for 99.1% / 99.5% / 98.9% of 4,489 player-seasons
Live-site test	2,707/2,707 player files return HTTP 200; 150 pages clicked through in a real browser with 0 errors
Code written	~780 lines Python (pipeline), ~660 lines TypeScript/TSX (website)

2 The starting plan

Before writing code we answered four questions. This is the framework to reuse for any project: **What must it do → What limits us → What shape should the system take → What tools fit that shape.**

2.1 Requirements (what you asked for)

1. Real, correct data, collected from the internet, not estimated.
2. Every player at every club in the top 5 leagues.
3. For each player: **cumulative career stats** split by competition, and **yearly stats** where each year is split by competition (league, Carabao/EFL Cup, FA Cup, Champions League, Super Cup, national team...).
4. Goal contributions and deeper stats: goals, assists, dribbles, and more.
5. Verify the data again after collecting it.
6. Website must work for anyone at any time, even with your laptop off; no errors.
7. Save everything in the `football-transfer-market` repository.

2.2 Constraints (what limits us)

Constraint	Consequence for the design
Most football stats sites block bots (Cloudflare "Just a moment..." pages, HTTP 403/405).	Pick a source that serves JSON to normal requests; don't fight anti-bot systems.
No source publishes "official" data for all 5 leagues + all cups + Europe in one place.	Use one consistent aggregator (Opta data via FotMob) and cross-check it with an independent one (Understat).
~2,700 players × many seasons × many competitions = tens of thousands of requests.	Cache every response on disk, run in parallel with a rate limit, make it resumable.
Site must work with your laptop off, free hosting, no server to maintain.	Pre-compute everything; serve static JSON files from Vercel's CDN. No live backend.
Limited tokens / time.	Build one example player end to end first, then run the same code for everyone.

2.3 The design decision: a "static data site"

There are two common shapes for a data website:

> **Option A: live backend (what the first version tried)**
> Browser → API server (FastAPI) → database (SQLite/Postgres). Needs a server running 24/7, costs money, breaks when the server/laptop is off; the first version even pointed at `127.0.0.1:8000` (your laptop).

> **Option B: pre-built static data (what we chose)**
> Pipeline builds JSON files once → files are deployed with the website → browser downloads only the file it needs. No server to crash, free, very fast, works with the laptop off. Trade-off: data updates when you re-run the pipeline.

Player stats change at most once per match day, so "rebuild and redeploy" is a perfect fit. This is the single most important architectural choice in the project.

2.4 The plan as a checklist

#	Step	Tool / language	Output
1	Find root cause of the wrong data	Reading old Python code	Found "projected 2024 → 2026" numbers
2	Test which data sources are reachable and complete	curl, Python	FotMob (primary), Understat (verification)
3	Write a cached, resumable scraper	Python + requests + threads	`cache/` raw JSON
4	Transform raw data into the shape the site needs	Python	`public/data/*.json`
5	Build one example (Haaland), inspect, fix issues	Python + screenshots	Validated structure
6	Rewrite the website to read the new data	TypeScript, React, Next.js, Tailwind	Home + player pages
7	Run for all players, verify against Understat	Python	`verification_report.json`
8	Browser-test pages, fix every error found	Playwright (Node.js)	0 errors
9	Commit, push, deploy preview, test, promote to production	Git, GitHub, Vercel CLI	Live site

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Requirements list	Wrote down your 7 wishes (real data, all players, career + season split by competition…) before coding.	Without a written target you can't tell when you're done or whether the result is right. A list beats a vague idea in your head.
Constraints table	Listed what limits us (bot blocks, cost, laptop off, time) and what each means for the design.	Constraints decide the architecture more than wishes do. Ignoring them is how the first version ended up depending on your laptop.
"Static data site" architecture	All data is computed in advance and shipped as files with the website.	Data changes once per match day and nobody writes to the site, so a live server + database (Option A) adds cost and failure points for no benefit.
One-example-first plan	Built Haaland's page end to end before running for 2,707 players.	Finding a mistake on 1 player takes minutes; finding it after a 1-hour run on everyone wastes the run.

3 Architecture: the big picture

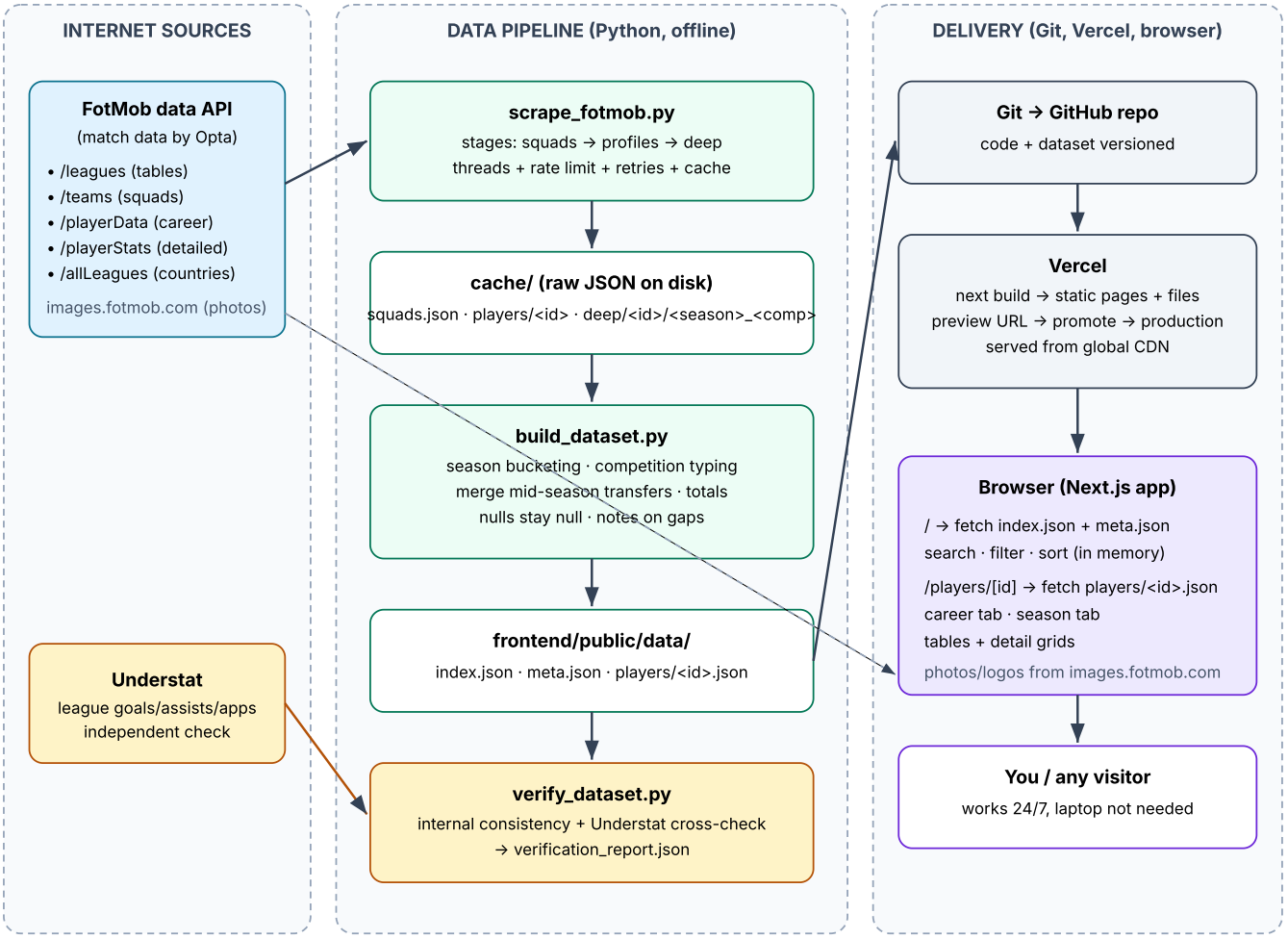


Figure 2. System architecture. Left: where data comes from. Middle: the offline pipeline. Right: how it reaches visitors.

Read the diagram like this

Everything in the middle column runs on a computer (this time, a cloud container) when you want fresh data. Its only output is a folder of JSON files. Those files are committed to Git and deployed with the website. When a visitor opens the site, no Python runs at all: the browser downloads small JSON files and React draws the tables.

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Three-zone design (sources → pipeline → delivery)	Kept collecting, processing and showing data in separate programs and folders.	Each part can be fixed or replaced alone. If FotMob changes, only the scraper changes; the website doesn't care.
Files as the hand-over point	The pipeline's only output is `public/data/*.json`; the website's only input is the same folder.	A simple, inspectable boundary. You can open any file and see exactly what the site will show.
CDN hosting	Vercel copies the files to servers worldwide.	Fast for every visitor, free tier, nothing to keep running.

4 Technology stack

4.1 Languages

Language	Where	Why this language
Python 3.13	Data pipeline: `scrape_fotmob.py`, `build_dataset.py`, `verify_dataset.py`	Best ecosystem for data work; simple HTTP + JSON; readable scripts.
TypeScript 5	Website logic: `src/lib/*.ts`, pages	JavaScript with types: catches mistakes like a misspelled field before the site runs.
TSX (React JSX)	Website UI: `page.tsx`, `PlayerView.tsx`, `Img.tsx`	Write HTML-like markup inside TypeScript; React turns it into the page.
CSS via Tailwind 4	Styling through class names like `bg-slate-900 rounded-2xl`	Style directly in markup; consistent spacing/colours; no separate CSS files to maintain.
JSON	The dataset and all configs	The common format both Python and the browser read natively.
Bash / shell	Running stages, git, Vercel CLI	Glue between tools.
JavaScript (Node.js)	Playwright browser tests (testing only)	Playwright's primary API.

4.2 Libraries, frameworks and services

Tool	Version	Role
requests (Python)	2.34	HTTP client for the scraper and verifier.
concurrent.futures / threading (Python stdlib)	built-in	Run up to 10 downloads at once; lock-based global rate limiter.
difflib, unicodedata (Python stdlib)	built-in	Fuzzy name matching and accent folding ("Mbappé" = "mbappe") in verification.
Next.js	16.3.3	React framework: routing (`/`, `/players/[id]`), building, optimisation.
React	19.2.8	UI library: components, state (`useState`), effects (`useEffect`), memoisation (`useMemo`).
Tailwind CSS	4	Utility-class styling.
ESLint + eslint-config-next	9 / 16.3.3	Code-quality rules (caught a React anti-pattern, see §14).
Geist font (next/font)	-	Self-hosted web font, no layout shift.
Playwright	1.56	Headless Chromium to screenshot and click through pages, record JS errors.
Git + GitHub	-	Version control and remote storage of code + data.
Vercel (+ Vercel CLI 64.5)	-	Build and host the site; preview and production deployments; global CDN.

4.3 Project file map (the parts that run the live site)

```
football-transfer-market/
└── project-material/
    ├── backend/pipeline/              ← DATA PIPELINE (Python)
    │   ├── scrape_fotmob.py           collect: squads → profiles → deep stats (cached)
    │   ├── build_dataset.py           transform: raw cache → website JSON
    │   ├── verify_dataset.py          verify: internal checks + Understat cross-check
    │   ├── verification_report.json   latest verification results
    │   ├── README.md                  how to run the pipeline
    │   └── cache/                     raw downloads (git-ignored, ~110 MB)
    └── frontend/                      ← WEBSITE (TypeScript / Next.js)
        ├── public/data/               the dataset the site reads
        │   ├── index.json             one row per player (1.2 MB)
        │   ├── meta.json              build time, season, counts
        │   └── players/<id>.json      2,707 full player files
        └── src/
            ├── app/layout.tsx         HTML shell, font, page title
            ├── app/globals.css        Tailwind import + dark theme colours
            ├── app/page.tsx           home: search, filters, player cards
            ├── app/players/[id]/page.tsx       route wrapper (server component)
            ├── app/players/[id]/PlayerView.tsx player page (client component)
            └── lib/
                ├── types.ts           TypeScript shape of the dataset (the "contract")
                ├── format.ts          number/money formatting, labels, stat groups
                └── Img.tsx            image that hides itself if missing
```

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Python for data, TypeScript for the website	Pipeline in `.py` files; site in `.ts/.tsx` files.	Each language is the strongest choice for its job: Python has the best data tools; browsers only run JavaScript, and TypeScript is JavaScript with safety checks.
Next.js instead of plain React or plain HTML	Pages live in `src/app/`; folders become URLs.	Gives routing, building and Vercel deployment for free. Plain HTML would need hand-written routing; Django/Flask would need a running server.
Tailwind CSS instead of separate CSS files	Styles written as classes, e.g. `rounded-2xl bg-slate-900`.	The old project already used it, it's quick to change, and the look stays consistent.
requests instead of a scraping framework (Scrapy) or a browser robot	Plain HTTP GET to JSON endpoints.	The data is already JSON, so no HTML parsing or fake browser is needed. Simplest tool that works.
Python standard library (threading, json, difflib)	Parallel downloads, file writing, name matching.	Built in, so there's nothing extra to install, and they're enough for this size.

5 Where we started: what was wrong

The first version (built with another agent) looked complete but the numbers were unreliable. Reading the code found the root causes. **Lesson: when data "looks wrong", read the code that produced it before collecting new data.**

Problem found	Evidence	Effect on the site
Projected, not real, stats	`fetch_api_football.py` : "free tier only allows up to 2024. We will fetch 2024 and mathematically project it to 2026", ages +2	Every 2026 number was an estimate.
Invented contract data	Release clauses, wages (default €35,000/week), buy-back clauses had no source	Financial figures were fiction.
Stale squads	Club list hard-coded for 2024/25 (Ipswich, Leicester, Southampton…)	Wrong clubs/leagues for many players.
Fake "sync" button	`/api/sync` returned "success … synchronized" whenever the backend was unreachable	Claimed updates that never happened.
Backend on your laptop	`BACKEND_URL` default `http://127.0.0.1:8000`	On Vercel this always failed silently and fell back to bundled data.
Unknowns turned into zeros	Schema defaults like `assists = Column(Integer, default=0)`	"0 assists" displayed where the truth was "unknown".
Only top scorers/assisters fetched	Endpoints `topscorers` , `topassists`	Most players had no real stats at all.
One 14 MB file loaded by every API call	`import rawPlayers from "@/data/players.json"`	Slow functions, hard to scale.

The general lesson

A system that silently "falls back" or "fills in" values is the most dangerous kind: it never shows an error, so you don't notice the data is wrong. Our rebuild follows one rule everywhere: **if the source doesn't have a number, store `null` and show "–"; never invent, never default to 0.**

🧰 What we used to diagnose, and why

Thing	How we used it	Why this, and not something else
Reading code before collecting new data	Opened the old `fetch_api_football.py` , `live_updater.py` and API routes.	Found the real cause (projected numbers) in minutes. Collecting new data on top of a broken process would have repeated the problem.
Root-cause table	Wrote each problem with its evidence and effect.	Turns "the data feels wrong" into a concrete fix list.
The null rule	Decided: unknown values are `null` and shown as "–".	A visible gap is honest; a fake 0 or estimate is a silent lie that nobody notices.

6 Data sources

6.1 How we chose: a reachability test

We sent one test request to every candidate source from the build machine and looked at the response code.

Source	Result	Decision
FBref	403, Cloudflare "Just a moment..." challenge	rejected anti-bot wall
Transfermarkt	405 (bot blocked)	rejected
Sofascore API	403	rejected
API-Football	403 without key; free tier limited to old seasons	rejected (root of the old problem)
Premier League official API	Requires pretending to be premierleague.com (fake Origin/Referer headers)	rejected blocked as bot-check circumvention; also PL-only
FotMob data API	200 JSON; full careers; every competition; detailed stats; Opta data	primary source
Understat	200 JSON; all players per league season	verification source

Ethics and terms of use

We only used sources that answer normal requests, sent requests at a polite rate, cached everything so each URL is fetched once, and never bypassed bot protection. FotMob's and Understat's terms may still restrict re-publishing their data on a public site; for a commercial product you would license data (e.g. from Opta/Stats Perform).

6.2 The FotMob endpoints we use

Endpoint	Calls	What we take from it
`/api/data/leagues?id=47` (PL), 87, 55, 54, 53	5	League table → the 20/18 club ids in each league; current season name.
`/api/data/teams?id=<club>`	96	Squad: every player id, name, position group.
`/api/data/playerData?id=<player>`	2,707	Bio (age, height, foot, nationality, contract), market value, **careerHistory**: every season × every competition with apps/goals/assists/rating, for club, national team and youth; **statSeasons**: list of season/competition pairs that have detailed stats.
`/api/data/playerStats?playerId=&seasonId=<entry>`	25,628	~50 detailed stats for one season-competition: minutes, xG, xA, shots, dribbles, chances, passes, duels, tackles, cards, goalkeeping...
`/api/data/allLeagues`	1	League → country, used to label "Bundesliga (Austria)" vs "Bundesliga (Germany)".

How we discovered these: open the public website, look at what JSON the page itself loads, then test the URL patterns with curl and inspect the keys with a short Python script. This "inspect before you build" step saved hours.

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
curl	Sent one request to each candidate source and read the status code (200 = OK, 403/405 = blocked).	Fastest possible test, a single command, before writing any code.
Browser developer tools (concept)	How the FotMob endpoints were found: a site's own pages load JSON you can see in the Network tab.	You discover real, stable data URLs instead of guessing.
Short Python inspection scripts	Loaded one response and printed its keys (`careerHistory` , `statSeasons` ...).	You must see the real shape of data before you design anything around it.
FotMob as primary source	Every player, every season, every competition, with detailed stats from Opta.	Reachable, complete, and consistent. The rejected sources were blocked, too narrow (PL only) or limited (old seasons).
Understat as a second source	Used only to check our numbers.	Collected independently by different people, so if both agree the number is very likely right.

7 Pipeline stage 1: collecting the data

File: `backend/pipeline/scrape_fotmob.py` (Python). It has three stages you run in order:

```
python scrape_fotmob.py squads                       # 5 leagues → 96 clubs → 2,707 player
ids
python scrape_fotmob.py profiles --workers 10        # 2,707 career profiles
python scrape_fotmob.py deep --workers 10 --seasons 4 # 25,628 detailed-stat entries
python scrape_fotmob.py deep --player 737066          # any stage, one player only (the
Haaland example)
```

7.1 The core algorithm: a cached, rate-limited, retrying fetch

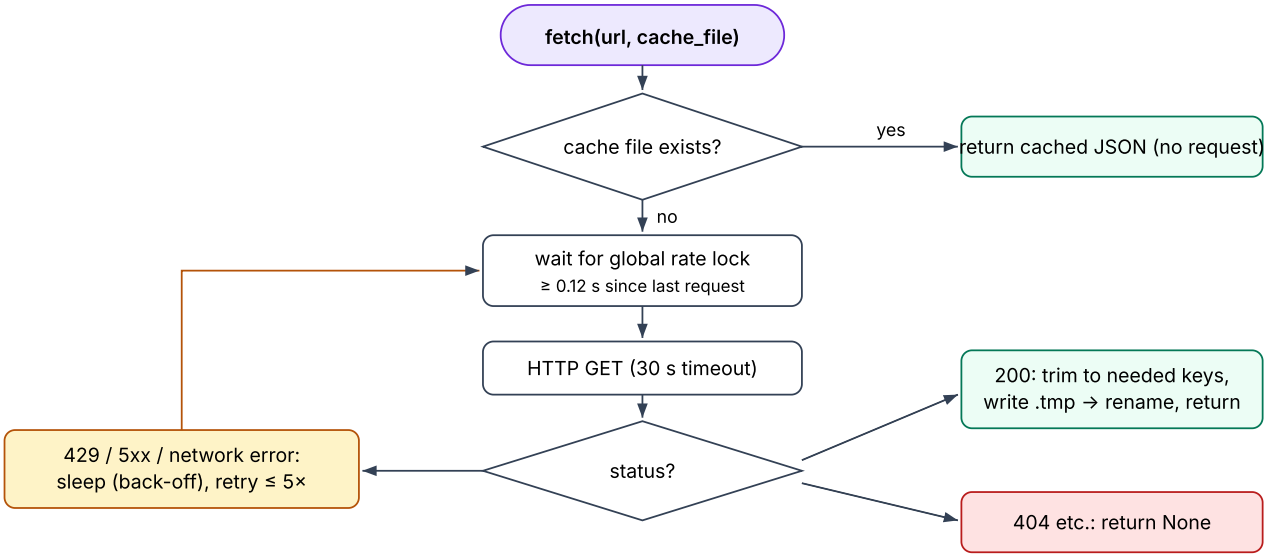


Figure 3. The fetch() function, the heart of the scraper. Every one of the ~28,400 requests goes through it.

7.2 Why each piece exists

Technique	Problem it prevents
Disk cache (one JSON file per request)	Re-downloading after a crash; hammering the source. A stopped run simply continues; a second run is instant.
Atomic write (write `.tmp`, then `os.replace`)	A half-written file if the process dies mid-write, which would otherwise be treated as valid cache.
Thread pool (10 workers)	Waiting on network latency one request at a time. First run did 0.7 profiles/s; with 10 workers ~8 requests/s (25,628 entries in ~50 min).
Global rate lock	Workers bursting together; keeps total rate polite no matter how many threads.
Retry with back-off on 429/5xx	Losing data to temporary server hiccups.
Trimming responses to needed keys	Disk usage: raw profile ~140 KB, deep stats ~130 KB (shot maps, heat maps). Trimmed cache is ~110 MB instead of ~4 GB.
Null-safe trimming	The run crashed at 25,603/25,628 because FotMob answered `null` for a few entries. Now an empty answer is stored as "no stats".
`--player` option	Testing on one player (Haaland) before spending ~1 hour on everyone.

7.3 The real code (simplified)

```
def fetch(path, params, cache_file, trim=None):
    if os.path.exists(cache_file):                # 1. cached? done
        return json.load(open(cache_file))
    for attempt in range(5):
        with _rate_lock:                          # 2. one request at a time leaves the
gate
            wait = _last_call[0] + MIN_INTERVAL - time.time()
            if wait > 0: time.sleep(wait)
            _last_call[0] = time.time()
        r = session.get(f"{BASE}/{path}", params=params, timeout=30)
        if r.status_code == 200:
            data = trim(r.json()) if trim else r.json()
            write_atomically(cache_file, data)    # 3. .tmp then os.replace
            return data
        if r.status_code in (429, 500, 502, 503, 504):
            time.sleep(5 * (attempt + 1)); continue  # 4. back off and retry
        return None                               # 5. 404: nothing there
```

7.4 Which detailed stats are fetched

The profile's `statSeasons` lists, for each season, the competitions with detailed data and an `entryId` like `"1-3"` (season #1, competition #3). The `deep` stage takes the 4 most recent seasons and every competition flagged `hasDeepStats`, and saves each as `cache/deep/<player>/<season>_<competitionId>.json`. Older seasons are skipped to keep the run to about an hour; apps/goals/assists for *all* seasons already come from the profile.

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Python `requests.Session`	One reusable connection per thread for all HTTP calls.	Reusing connections is faster than opening a new one per request.
Disk cache (JSON files)	Every answer saved under `cache/`; checked first before any request.	Crash-proof and polite: re-running costs nothing. A database would be overkill for "save this answer".
`ThreadPoolExecutor` (10 workers)	Ran many downloads at the same time.	Downloads spend most time waiting for the network, which threads handle well. `asyncio` would also work but needs more complex code.
`threading.Lock` rate limiter	All threads pass through one gate that enforces a minimum gap between requests.	Keeps the total speed polite however many threads run. Without it, 10 threads could burst and get blocked.
Retries with back-off	On 429/5xx errors, wait longer each time and try again (max 5).	Servers have brief hiccups. Giving up immediately loses data; retrying instantly makes it worse.
Atomic write (`.tmp` + `os.replace`)	Write to a temporary file, then rename it in one step.	A crash mid-write can never leave a broken file that later looks like valid cache.
`argparse` command-line options	`squads` / `profiles` / `deep` stages, `--workers`, `--seasons`, `--player`.	Run one stage or one player for testing without editing code.

8 Pipeline stage 2: transforming the data

File: `backend/pipeline/build_dataset.py`. Input: the cache. Output: `frontend/public/data/`. This is where most of the "thinking" lives, because raw data is messy in ways you only discover by looking at it.

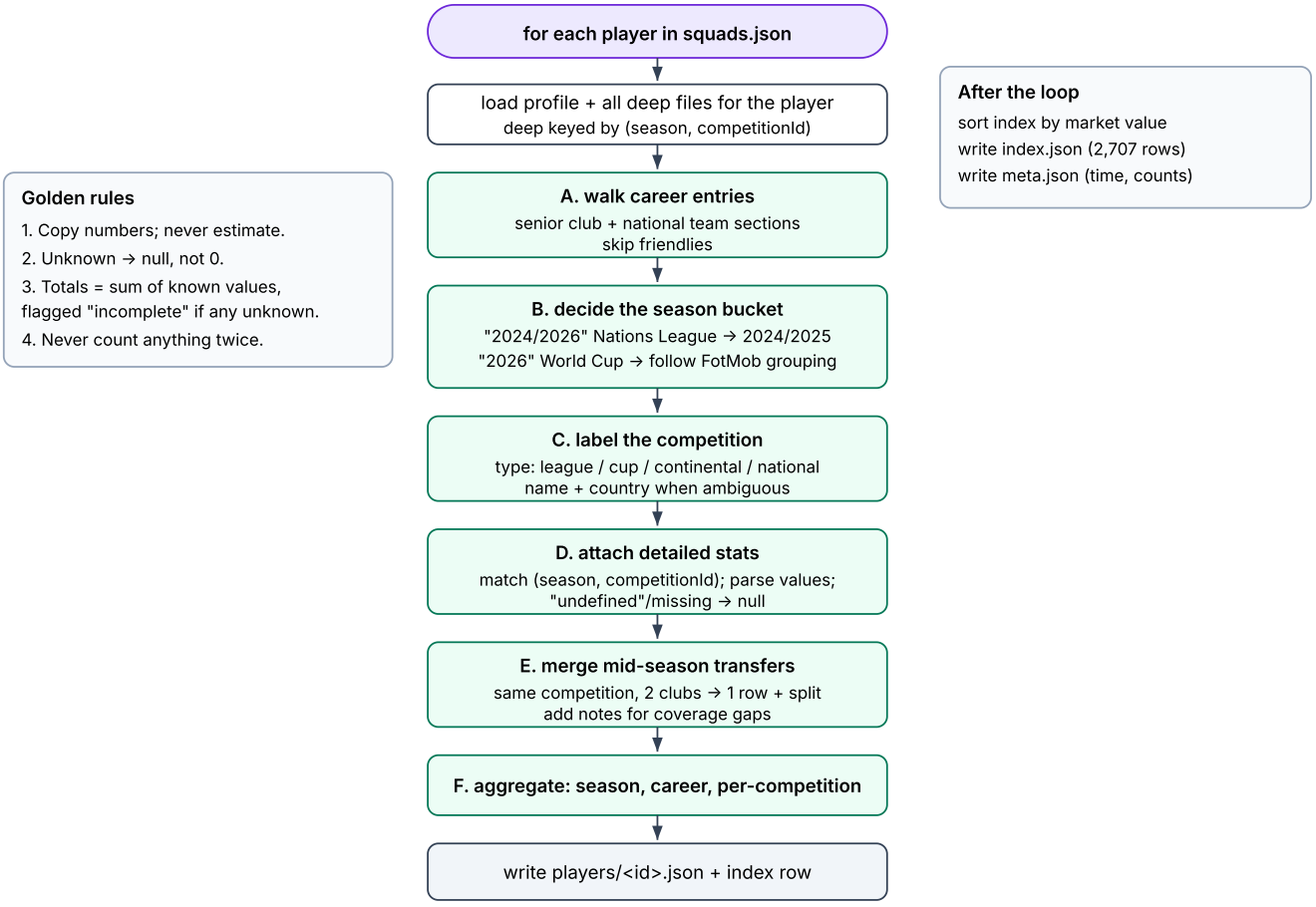


Figure 4. build_dataset.py, per player.

8.1 Step B: season bucketing (the trickiest rule)

Club seasons are named `2025/2026`, but other entries are named differently:

Raw season name	Example	Bucket we use	Why
`2025/2026`	Premier League	2025/2026	Normal club season.
`2025` inside a club entry	FIFA Club World Cup 2025	2025/2026 (the club entry's season)	FotMob already files it under the club season.
`2026` (national team)	World Cup 2026	2026/2027	Follow the season in which FotMob lists that tournament's detailed stats, so basic and detailed numbers land together.
`2024/2026` (national team)	Nations League 2024/25	2024/2025	FotMob labels some editions over two years; they are played in the first club season. Without this rule we showed fake seasons like "2020/2022".
`2018` (club)	Norwegian league (Molde)	2018	Calendar-year leagues stay as they are.

8.2 Step C: labelling competitions

- **Type** by name patterns: continental (*Champions League, Europa, Conference, UEFA Super Cup, Club World Cup…*), domestic cup (*Cup, Copa, Coppa, Pokal, Coupe, Shield, Supercopa, …mesterskap…*), national team (by section), otherwise league.
- **Name** from FotMob's league list. If the same name exists in several countries ("Bundesliga", "Premier League", "Super Cup", "Cup") the country is appended: *Bundesliga (Austria)* vs *Bundesliga (Germany)*.

8.3 Step E: merging mid-season transfers

Dean Huijsen, 2023/24: FotMob's career lists *Juventus, Serie A, 1 app* and *Roma, Serie A, 13 apps*, but the detailed Serie A stats cover the whole season (14 apps). Attaching those detailed stats to both rows counted them twice. Fix: rows with the same competition in the same season merge into one row, *"Roma / Juventus: 14 apps, 2 goals"*, with a per-club split kept underneath. This halved the career-vs-detailed disagreements and matches Understat (which reports the combined 14 apps, 2 goals).

8.4 Step F: how totals are calculated

Kind of stat	Examples	How it is combined
Counts	goals, assists, minutes, shots, dribbles, tackles	Sum of known values.
Rates and percentages	rating, pass accuracy %, dribble success %, save %	Minutes-weighted average (a 90-minute game counts more than a 5-minute cameo). Never summed.
Unknown parts	Old seasons where assists = "undefined"	Total = sum of known values, field listed in `incomplete`; the site shows a * next to it.

```
# the "unknown is not zero" rule, from build_dataset.py
def num(v, as_float=False):
    if v is None or v == "" or v == "undefined":
        return None                       # unknown stays unknown
    f = float(str(v).replace(",", "").replace("%", ""))
    return f if as_float else int(round(f))
```

8.5 Notes added to rows

- *"Detailed stats cover 11 of 16 matches (qualifying / early rounds have no detailed data)"*: e.g. Europa League including qualifiers. The headline apps/goals/assists still include qualifiers.
- *"Match data shows 1 goal… the source's season summary differs"*: the two FotMob endpoints disagree for the same matches (usually the current season, before one side updates). We show both rather than silently picking one.

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Python dictionaries and lists	Grouped entries by season and by competition (`OrderedDict` , `defaultdict`).	Grouping and summing is the core job, and dictionaries are Python's built-in tool for it. pandas would work but hides the rules we needed to control precisely.
Regular expressions (`re`)	Recognised season formats (`2024/2026` , `2026`) and competition types ("cup", "pokal", "coppa"...).	Short patterns that cover many spellings at once.
A lookup table from FotMob's league list	Added the country to ambiguous names (Bundesliga (Austria) vs (Germany)).	Uses the source's own data instead of a hand-made list that would go out of date.
Merge step for mid-season transfers	Same competition and season at two clubs → one row with a per-club split.	Prevents double counting, which was proven by Understat agreeing with the combined numbers.
Sum vs weighted average	Counts are summed; ratings and percentages are averaged by minutes played.	Adding up percentages gives nonsense (70% + 80% ≠ 150%).
Notes on rows	Text caveats stored with the data and shown on the site.	Explains real gaps (qualifiers) instead of hiding them or guessing.

9 The data model

The website reads three kinds of files. The same shape is written down twice: produced by Python (`build_dataset.py`) and described in TypeScript (`src/lib/types.ts`). That TypeScript file is the **contract** between pipeline and site: if they disagree, TypeScript reports an error at build time.

players/737066.json (one player)

```
{ id, name, team, team_id, league,
position, nationality, age, height_cm,
preferred_foot, contract_end,
market_value: {value_eur, date, source},
career_totals: {
appearances, goals, assists, incomplete[],
detailed: {minutes, xg, dribbles, …},
detailed_seasons: ["2026/2027", …] },
career_by_competition: [
{ competition, type, seasons,
appearances, goals, assists,
detailed:{…} }, … ],
seasons: [
{ season: "2025/2026", teams: [..],
totals: {apps, goals, assists},
totals_detailed: {…},
competitions: [
{ competition, type, team, edition,
appearances, goals, assists, rating,
detailed: {~50 stats} | null,
by_team: [...] | null, notes: [] }
] }, … ],
source: {provider, url} }
```

index.json (all players, small rows)

```
[ { id, name, team, team_id, league,
position, position_group, nationality,
age, market_value_eur,
current_season: {apps, goals, assists, minutes},
last_season: {…}, last_season_name,
career: {apps, goals, assists} }, … ]
```

Enough for the home page cards, search, filters and sorting; 1.2 MB for 2,707 players.

meta.json

```
{ built_at, current_season, players,
clubs, leagues: {…}, source }
```

Why split into files?

Home page downloads 1 small file, not 57 MB.
A player page downloads only that player (~10–50 KB).
The CDN caches each file close to the visitor.

Figure 5. Shape of the dataset. Green = career level, blue = per season per competition.

9.1 Example: Haaland, 2025/26, as stored

Competition	Type	Apps	Goals	Assists	Minutes	Shots	Dribbles
Premier League (England)	league	35	27	8	2,958	126	17
FA Cup (England)	domestic_cup	4	3	1	231	6	1
EFL Cup	domestic_cup	3	0	0	199	4	2
Champions League	continental	10	8	0	755	35	1
FIFA Club World Cup	continental	4	3	1	255	15	2
World Cup Qualification UEFA	international	8	16	2	706	41	5
2025/2026 total		**64**	**57**	**12**	**5,104**	**227**	**28**

Premier League row cross-checked with Understat: 35 games, 27 goals, 8 assists, an exact match.

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
JSON files split three ways	`index.json` for lists, one file per player, `meta.json` for info.	Each page downloads only what it needs: a 1.2 MB list instead of 57 MB.
TypeScript interfaces as a contract	`src/lib/types.ts` describes every field the site reads.	If the data shape changes, the build fails loudly instead of the page breaking silently for users.
`null` instead of 0 for missing values	Stored as JSON `null`, shown as "–".	Keeps "unknown" and "zero" different, which they are.

10 Pipeline stage 3: verification

File: `backend/pipeline/verify_dataset.py`. "It looks right" is not verification. We check the data two ways and write the results to `verification_report.json`.

10.1 Internal checks (hard errors fail the run)

- Every season's totals equal the sum of its competitions.
- No negative appearances, goals or assists.
- Every player in the index has a file. (Separate check: all 2,707 files have every field the website needs.)
- Counted, not failed: where FotMob's two endpoints disagree on goals/assists (now explained by row notes).

Result: 0 hard errors.

10.2 Independent cross-check with Understat

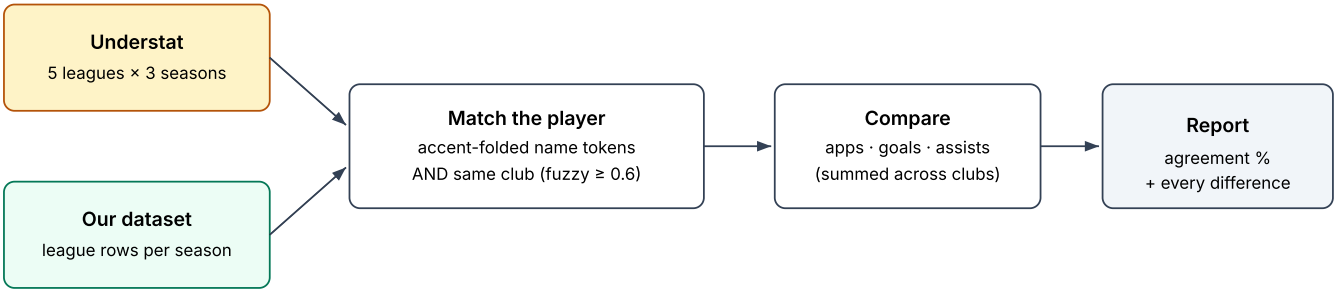


Figure 6. Cross-check algorithm.

Measure	Result
Player-league-seasons compared (2023/24, 2024/25, 2025/26)	4,734, of which 4,489 matched by name + club
Appearances agree	4,449 / 4,489 = **99.1%**
Goals agree	4,467 / 4,489 = **99.5%**
Assists agree	4,440 / 4,489 = **98.9%**

10.3 What the differences turned out to be

- **±1 assist** (Saka 9 vs 8, Damsgaard 10 vs 9): the two sources define assists differently. Opta's definition (FotMob) is the one leagues publish.
- **Big gaps at first** (Vitinha PSG vs Vitinha Marseille, two "Toni Martínez"): our matcher paired different people with the same name. Fix: require the same club as well. Big gaps went from 7 to 1.
- **The last one** (Amad Diallo 2023/24: 9 apps vs 1): Understat's record is incomplete; 9 is correct.

> **Lesson**
> Verification can be wrong too. When a check fails, investigate whether the data or the check is at fault. Two of our "data errors" were actually bugs in the verifier and in the builder.

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Independent source comparison	Compared 4,489 player-seasons with Understat.	The only strong evidence that numbers are right: two independent collectors agree.
Accent folding (`unicodedata`)	"Mbappé" → "mbappe" before comparing names.	Different sources spell names differently. Without this, matches are missed.
Fuzzy matching (`difflib`) + club check	Name similarity plus "same club" to pair players.	Name alone matched the wrong Vitinha. The club makes the match reliable.
Invariant checks	Season total = sum of its competitions; no negatives.	Cheap rules that catch whole classes of bugs automatically.
A JSON report file	All results and every difference saved to `verification_report.json`.	You can inspect and compare runs later instead of trusting a printout.

11 The frontend: how the website works

11.1 Next.js App Router in 60 seconds

In Next.js, **folders are URLs**. `src/app/page.tsx` is `/`; `src/app/players/[id]/page.tsx` is `/players/anything`, with the square brackets meaning "a variable part of the URL". Files in `public/` are served as they are, so `public/data/index.json` is available at `/data/index.json`.

Components are **server components** by default (rendered on the server/at build time). A file starting with `"use client"` runs in the browser and can use state, clicks and effects. We use:

File	Kind	Job
`app/layout.tsx`	server	HTML skeleton, Geist font, page title "Football Transfer Market".
`app/page.tsx`	client	Home: loads index + meta, search box, league/position/club filters, sort, cards, "Show more".
`app/players/[id]/page.tsx`	server	Reads the `id` from the URL (in Next.js 16 `params` is a Promise: `await params`) and renders PlayerView.
`app/players/[id]/PlayerView.tsx`	client	Fetches the player file; header, KPI cards, two tabs, tables, expandable detail grids.
`lib/types.ts`	types	Shape of every JSON file (the contract).
`lib/format.ts`	helpers	`n()` number formatting with "–" for null, `money()`, image URLs, labels, detailed-stat groups.
`lib/Img.tsx`	client	Image that hides itself when the photo/logo doesn't exist (no broken-image icons).

11.2 Home page logic

```
const [rows, setRows]   = useState<IndexRow[] | null>(null);   // data
const [query, setQuery] = useState("");                         // UI state
const [league, setLeague] = useState("All");  // … position, club, sort, limit

useEffect(() => {                        // runs once after first render
  fetch("/data/index.json").then(r => r.json()).then(setRows);
}, []);

const filtered = useMemo(() => {         // recomputed only when inputs change
  return rows.filter(r => matchesLeague && matchesPosition && matchesClub && matchesSearch)
             .sort(bySelectedKey);
}, [rows, query, league, position, club, sort]);

filtered.slice(0, limit).map(r => <Link href={`/players/${r.id}`}>…card…</Link>)
```

- **Why filter in the browser?** 2,707 rows is small; filtering in memory is instant and needs no server.
- **Accent-insensitive search:** `s.normalize("NFD").replace(/[̀-ͯ]/g, "")` makes "mbappe" find "Mbappé".
- **"Show more" (60 at a time):** rendering 2,707 cards with photos at once would be slow.

11.3 Player page logic

- **Career tab:** `career_by_competition` grouped by type (League → Domestic cup → Continental → National team), plus an "All competitions" total row.
- **Season tab:** one collapsible card per season (summary line: apps, goals, assists, minutes); opening it shows a table with a row per competition, the per-club split for transfers, and amber notes for coverage gaps.
- **One table component for both** (`StatTable`): columns Apps, Min, Goals, Assists, xG, xA, Shots, Dribbles, Chances, Tackles, Rating. Clicking a row with ▼ opens `DetailGrid`: all ~50 stats grouped into Playing time, Attacking, Creating, Carrying & duels, Defending, Goalkeeping, Team while on pitch, Discipline.
- **Honest display:** null → "–"; incomplete totals → "*"; footnote explaining which seasons have detailed stats; a caption on the career tab saying the detailed columns start in 2023/24; goalkeepers told minutes aren't published.
- **Edge cases handled:** invalid id → "Player not found"; 45 players with no senior games → a clear message instead of an empty table.

11.4 Styling

Tailwind classes directly in the markup, dark theme (`bg-slate-950` background, emerald accent), responsive grids (`sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4`), and wide tables that scroll sideways on phones with the competition column sticky (`sticky left-0`).

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
`"use client"` components	Home page and player view run in the browser.	They need clicks, typing and state. Server components can't do that.
`useState`	Stores the loaded data, search text, selected filters, open tab, open row.	When state changes, React redraws the page automatically, with no manual page updates.
`useEffect`	Downloads the JSON once, after the page first appears.	The standard place for "do this when the page loads".
`useMemo`	Re-filters and re-sorts 2,707 players only when a filter actually changes.	Avoids redoing the same work on every tiny redraw.
`fetch()`	Gets `/data/index.json` and `/data/players/<id>.json`.	Built into every browser, so no library is needed.
`next/link`	Player cards link to `/players/<id>`.	Moves between pages without a full reload, so it's faster.
Reusable components (`StatTable`, `DetailGrid`, `Img`, `Kpi`)	One table component used for both the career and season views.	Write once, fix once. The old 947-line single file was hard to change.
Tailwind responsive classes	`sm:`, `lg:`, `xl:` prefixes change the layout by screen width.	One page works on phone and desktop without separate designs.

12 Hosting and deployment

12.1 Git and GitHub

Branch	What it holds
`main`	README + LICENSE (initial commit).
`claude/import-deployed-app`	The first site, downloaded from the old Vercel deployment.
`project-material`	Everything your other agent pushed (old backend, data, handoff docs).
`claude/data-pipeline-rebuild`	The rebuild (same content as v2; older commits show "Unverified").
`claude/data-pipeline-v2`	**Current version**: pipeline + dataset + new site, all commits verified.

What is *not* committed (`.gitignore`): `node_modules/` , `.next/` , `.vercel/` , `.env*` (secrets), the raw `cache/` (110 MB, re-creatable) and logs.

12.2 Vercel: preview → production

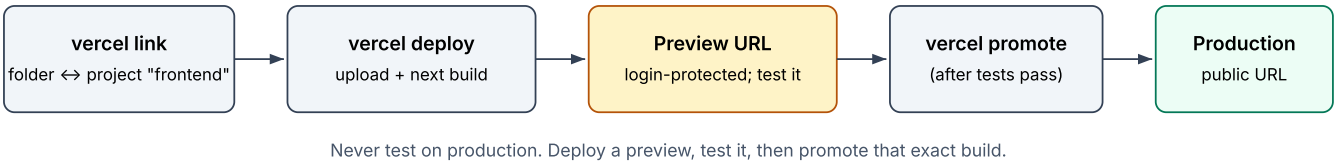


Figure 7. Deployment flow used for the live site.

- **Build output:** `/` is pre-rendered static HTML; `/players/[id]` is rendered on demand; all JSON files are static assets on the CDN.
- **Preview protection:** preview URLs redirect (302) to a Vercel login, so we tested them with `vercel curl` , which authenticates as you.
- **Why it works with your laptop off:** nothing points to your machine anymore; everything lives on Vercel.
- **History:** three deploy attempts from your machine six days earlier had failed (status Error); production had stayed on a 12-day-old build until our promotion.

12.3 Testing before handing over

Test	Tool	Result
Lint (code-quality rules)	ESLint	0 problems
Type check + production build	TypeScript, `next build`	Compiled successfully
Shape of all 2,707 player files	Python script	All fields present; found 45 players with no seasons → fixed UI
Browser sweep (home interactions + 150 player pages, clicking tabs and rows)	Playwright / Chromium	150/150, 0 JS errors, 0 failed requests (local and live)
Every player file on the live site	curl × 2,707 (16 in parallel)	2,707 × HTTP 200

🧰 What we used here, how, and why only this

Thing	How we used it	Why this, and not something else
Git commits and branches	Saved each step as a commit on `claude/data-pipeline-v2`.	You can see, undo or compare any change. Working on a branch keeps `main` safe.
`.gitignore`	Excluded `node_modules`, build output, secrets (`.env*`), the raw cache.	Those files are huge, re-creatable or secret. Committing them bloats or leaks the repo.
Vercel CLI (`vercel link / deploy / promote`)	Linked the folder to your project, deployed a preview, then promoted it.	Repeatable from the terminal. The dashboard works too but is harder to automate.
Preview before production	Tested the private preview URL first.	Visitors never see a broken version.
ESLint, TypeScript, `next build`	Ran before every deploy.	Catch mistakes on your machine, not in front of users.
Playwright	A robot browser opened 150 pages, clicked tabs and rows, and recorded any errors.	Tests what a real visitor does. Reading code can't prove the page works.

13 End-to-end: opening a player page

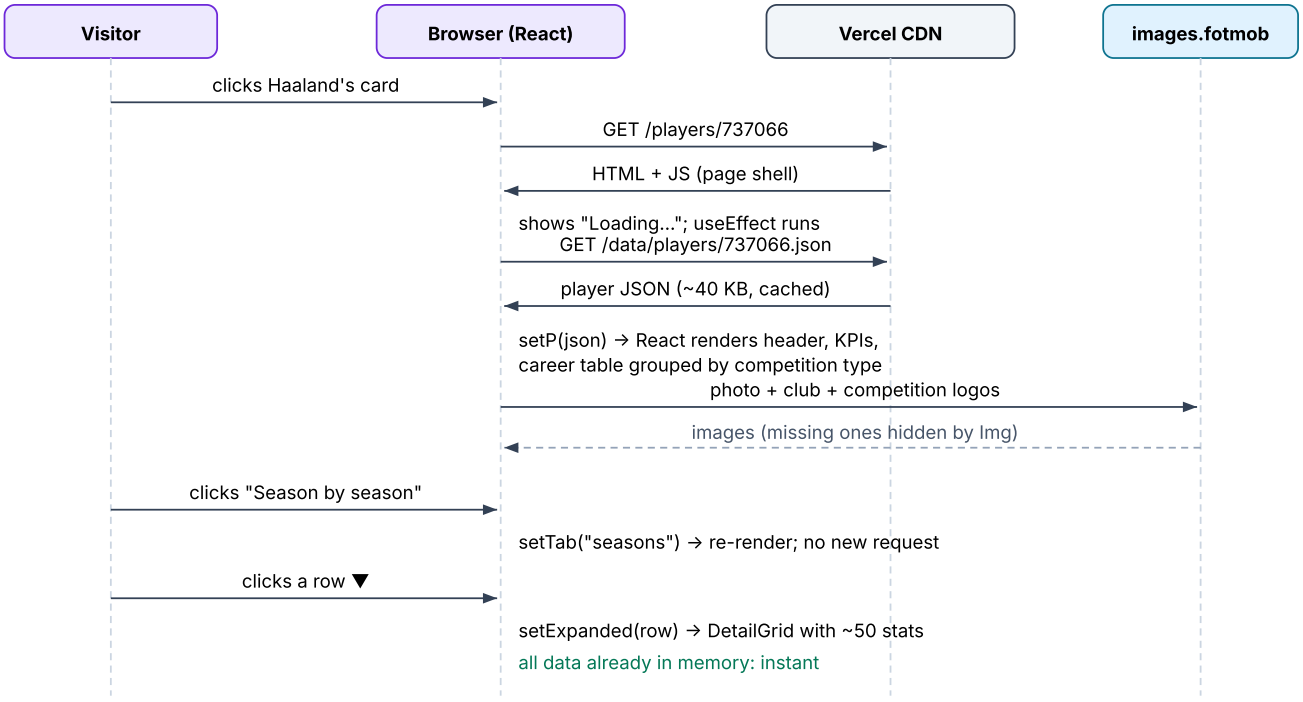


Figure 8. Sequence of events when a visitor opens a player and explores it.

Note what is **absent**: no database query, no API server, no Python. The heavy lifting was done by the pipeline hours earlier. That's why the page is fast and can't "go down" when a server or laptop is off.

13.1 The full data journey of one number

Haaland's 27 Premier League goals in 2025/26:

1. Opta records each goal at the match. FotMob shows Opta data.
2. `scrape_fotmob.py profiles` saves his profile: `careerHistory → senior → seasonEntries["2025/2026"] → tournamentStats[leagueId 47].goals = "27"`.
3. `scrape_fotmob.py deep` saves `cache/deep/737066/2025-2026_47.json` with goals 27, xG 25.81, shots 126, minutes 2,958....
4. `build_dataset.py` parses "27" → 27, buckets it into 2025/2026, labels it "Premier League (England)", type league, attaches the detailed stats, adds it to the season total (57) and to his career Premier League total (121).
5. `verify_dataset.py` fetches Understat EPL 2025: Erling Haaland, Manchester City, 27 goals. Match.
6. Committed in `public/data/players/737066.json`, deployed to Vercel.
7. The browser fetches the file and `n(27)` prints "27" in the Goals column.

🧰 How to study a system like this

Thing	How we used it	Why this, and not something else
Sequence diagram	Drew who talks to whom, in time order, when a page opens.	The clearest way to understand (and debug) the flow of a request.
Tracing one number end to end	Followed Haaland's 27 goals from match to screen.	If you can trace one value through every step, you understand the whole system.

14 Problems we hit and how we solved them

These are the real problems from this project, in the order they happened. Reading this table is probably the fastest way to learn what goes wrong in data projects.

#	Problem	How we noticed	Fix / how it's avoided
1	Old stats were projections	Read the scraper code	Rebuilt from a source with real current data; rule "never estimate".
2	Main stats sites block bots	403/405 reachability test	Chose sources that serve normal requests; no bypassing.
3	Network policy blocked vercel.com at first	Fetch failed (ENOTFOUND / 403)	Allowed the domain in environment settings.
4	Scraping too slow (0.7/s)	Progress log	10 threads + shared rate lock; resumed thanks to cache.
5	Same league name in different countries	Haaland showed "Bundesliga" twice	Country from FotMob league list appended when ambiguous.
6	One unknown value wiped whole totals	Career assists showed "unknown"	Sum of known values + "incomplete" flag + "*".
7	Norwegian cup classified as league	Example output	Extended cup name patterns.
8	Fake seasons "2024/2026", "2020/2022"	Screenshot	Multi-year national labels bucketed into their first season.
9	Team stats shown as goalkeeping stats for a striker	Screenshot	New group "Team while on pitch".
10	Goalkeeper field names guessed wrong	Inspected a keeper file	Mapped real ids (penalty_saves, ...). Lesson: inspect real data, don't guess.
11	React lint error: setState directly in useEffect	ESLint	Computed "invalid id" during render instead.
12	`PageProps` type "not found"	tsc	Next.js generates these types during build; ran `next build`.
13	Verifier matched wrong people with same name	Huge differences (Vitinha)	Match on name AND club.
14	Scraper crashed at 99.9% on `null` responses	Traceback in log	Null-safe trimming; rerun fetched only the 25 missing.
15	Mid-season transfers double-counted detailed stats	447 internal mismatches; Understat sided with combined numbers 79/82	Merge same-competition rows per season; keep per-club split.
16	Qualifiers not in detailed stats	Remaining mismatches analysed	Keep full numbers, add a visible note on the row.
17	Career minutes looked wrong (246 apps, 2,158 min)	Screenshot (Mbappé)	Caption: detailed columns cover 2023/24 onwards.

18	No minutes for goalkeepers	Inspected keeper data	Left as "–" (no estimating) and explained on the page.
19	Local test server served a stale build	404/500 in tests	Stop old server before restarting; test against a fresh build.
20	Broken-image icons for missing photos	Pre-handover review	`Img` component hides failed images.
21	45 players with no senior games → empty tables	Shape check of all files	Clear "no appearances yet" message.
22	Partial data almost committed to Git	Git hook warning	Moved partial output out; committed only the verified final build.
23	A GitHub token was pasted into chat	Visible in conversation	Token revoked immediately; tokens belong in environment variables, never in chat or code.
24	Commits shown as "Unverified"	Git hook	Fresh branch `claude/data-pipeline-v2` with verified identity (no history rewrite needed).

15 Other folders in the repository

The `project-material/` folder also contains parts of the first version. They are **not used by the live site**, but they're worth understanding because they show a different ("bigger") architecture.

Folder	Language / tech	What it is	Status
`backend/main.py`, `models.py`, `schemas.py`, `database.py`	Python, FastAPI, SQLAlchemy, Pydantic, SQLite	REST API (`/players`, `/players/{id}`, `/stats/summary`) over a SQLite DB with tables `players` and `player_competition_stats`.	replaced by static JSON
`backend/*.py` (~55 scripts)	Python	Earlier data fixes: enrich, dedupe, "brutal" verification loops, valuation.	legacy
`backend/sql_app.db`, `real_data.csv`, `raw_data_*.txt`	SQLite / CSV / text	Old dataset.	legacy
`ingestion/ingest.py`	Python, requests, pandas, boto3	API-Football → AWS S3 loader.	never run in production
`ml_model/train.py`	Python, XGBoost, scikit-learn, pandas	Market-value model trained from a Postgres feature table.	not used
`dbt/`	dbt (SQL transformations)	Project config only; no models written.	scaffold
`terraform/`	Terraform (HCL), AWS	Infrastructure as code: ECS cluster, RDS Postgres, load balancer, S3 bucket.	never deployed

When would you use that bigger architecture?
When data changes every few seconds (live scores), users write data (accounts, comments), or the dataset is far too big to ship as files. For read-only stats refreshed daily, it adds cost and failure points without benefit. Start simple; add a database and API only when a requirement forces you to.

16 How to run, update and extend it

16.1 Refresh the data (e.g. weekly)

```
git clone https://github.com/ssupreethsg/football-transfer-market.git
cd football-transfer-market && git checkout claude/data-pipeline-v2
cd project-material/backend/pipeline
pip install requests

rm -rf cache/players cache/deep cache/teams cache/leagues   # force fresh downloads
python scrape_fotmob.py squads
python scrape_fotmob.py profiles --workers 10
python scrape_fotmob.py deep --workers 10 --seasons 4
python build_dataset.py
python verify_dataset.py            # must print "hard errors: 0"
```

16.2 Run the website locally

```
cd ../../frontend
npm ci                 # install exact dependency versions
npm run dev            # http://localhost:3000, live reload
npm run lint && npm run build   # what must pass before deploying
```

16.3 Deploy

```
vercel link --project frontend      # once
vercel deploy                       # preview; test it
vercel promote <preview-url>        # make that build live
```

16.4 Ideas for extending it

- **Automatic weekly refresh:** a GitHub Actions workflow that runs the pipeline, commits the new data, and Vercel redeploys.
- **More history:** `--seasons 8` for detailed stats further back (longer run).
- **Charts:** goals per season as a bar chart on the player page.
- **Compare two players** side by side (both files are already separate JSONs).
- **Transfers page:** FotMob league pages also contain transfers; a new pipeline stage + page.

17 Lessons for your next project

17.1 A planning template (copy this)

Question	This project's answer
1. What exactly must the user see/do?	Search players; see career + season stats split by competition.
2. Where does the data come from, and is it allowed and reachable?	FotMob (Opta), Understat; tested with curl first.
3. How often does it change?	After each match day → batch rebuild is fine.
4. Does anyone write data?	No → no database or backend needed.
5. What's the smallest end-to-end slice?	One player (Haaland), raw data to screen.
6. How will I know it's correct?	Independent source + internal invariants + browser tests.
7. Where will it run, and what if my laptop is off?	Vercel CDN; laptop irrelevant.

17.2 Data and pipeline checklist

- Look at real raw responses before writing transform code (several bugs came from guessing field names).
- Cache every request; make every stage resumable; write files atomically.
- **Unknown ≠ 0.** Keep `null`, show "–".
- Never let a fallback silently pretend to succeed.
- Watch for double counting when combining two views of the same data.
- Separate counts (sum) from rates (weighted average).
- Verify against an independent source, and verify the verifier.
- Build one example end to end before scaling up.

17.3 Frontend checklist

- Write the data shape as TypeScript types first; let the compiler check you.
- Keep files small: one big file (the old 947-line page) is hard to fix; we split into page, view, table, grid, helpers.
- Load only what a page needs (index vs one player file).
- Design every state: loading, error, not found, empty, missing image.
- Read your framework version's docs; Next.js 16 changed APIs (e.g. `params` is a Promise).
- Run lint, build and a real-browser click-through before every deploy.

17.4 Deployment and security checklist

- Deploy to preview, test, then promote. Never debug on production.
- Never paste tokens or passwords into chats, code or commits; use environment variables; revoke anything exposed.
- Use fine-grained tokens limited to one repository, with short expiry.
- Don't commit generated or partial data; commit the verified final output.
- Don't point production at `localhost`.

18 Glossary

Term	Meaning
API	A URL that returns data (usually JSON) for programs instead of a web page for people.
JSON	Text format for data: `{"name": "Haaland", "goals": 27}`.
Scraper	Program that downloads data from websites/APIs automatically.
Cache	Saved copy of a download so it doesn't need to be fetched again.
Rate limit	Maximum request speed, to be polite and avoid being blocked.
Pipeline	Chain of steps that turns raw input into finished output (collect → transform → verify).
Static site	A site whose files are prepared in advance; the server just hands them out.
CDN	Network of servers worldwide holding copies of your files near visitors (Vercel provides one).
Next.js	React framework adding routing, building and hosting conventions.
React component	Function returning UI; re-runs when its state changes.
useState / useEffect / useMemo	React hooks: store values / run code after render (e.g. fetch) / cache a computed result.
Server vs client component	Runs on the server (no interactivity) vs in the browser (`"use client"`, clicks and state).
TypeScript type	Description of a data shape the compiler checks your code against.
Tailwind	CSS written as small class names in the markup.
Lint	Automatic check for code mistakes and bad patterns.
Preview / production deployment	A private test copy of the site / the public live site.
Branch / commit / push	Line of work / saved snapshot / upload snapshots to GitHub.
Opta (Stats Perform)	Company that records match events; the data source behind FotMob and the leagues.
xG / xA	Expected goals / expected assists: chance quality, from 0 to 1 per shot or pass.
Null	"No value / unknown". Different from 0.

19 How and where to learn all this

You don't need to learn everything at once. Learn in this order, and build something small after each step. All resources below are free unless marked.

19.1 What to learn, in order, and where

#	Topic	Why you need it (in this project)	Where to learn (free)	Time
1	Using a computer like a developer: terminal, folders, VS Code	Every command in this guide runs in a terminal.	The Odin Project, "Foundations" (theodinproject.com) · VS Code docs (code.visualstudio.com/docs)	1 week
2	Git and GitHub	Saving and sharing your work; deploying from GitHub.	learngitbranching.js.org (interactive) · GitHub Skills (skills.github.com) · Pro Git book, ch. 1–3 (git-scm.com/book)	1 week
3	Python basics	The whole data pipeline.	CS50's Introduction to Programming with Python (cs50.harvard.edu/python) · Automate the Boring Stuff (automatetheboringstuff.com) · official tutorial (docs.python.org/3/tutorial)	4–6 weeks
4	HTTP, APIs and JSON	How the scraper talks to FotMob; how the site loads data.	MDN "An overview of HTTP" and "Working with JSON" (developer.mozilla.org) · requests quickstart (requests.readthedocs.io)	1 week
5	Working with data in Python	Grouping, summing, cleaning, comparing.	Kaggle Learn: Python + Pandas (kaggle.com/learn) · Python docs for `json`, `collections`, `re`	2–3 weeks
6	HTML, CSS and JavaScript	What the browser actually runs.	MDN Learn web development (developer.mozilla.org/en-US/docs/Learn) · javascript.info · freeCodeCamp Responsive Web Design (freecodecamp.org)	4–6 weeks
7	TypeScript	Types that catch data-shape mistakes.	TypeScript Handbook (typescriptlang.org/docs/handbook) · "TypeScript for JavaScript Programmers"	1–2 weeks
8	React	Components, `useState`, `useEffect`, `useMemo`.	react.dev/learn (official, interactive; do "Quick Start" then "Thinking in React")	2–3 weeks
9	Next.js (App Router)	Pages, routing, building, deploying.	nextjs.org/learn (official course) · Next.js docs (nextjs.org/docs)	2 weeks
10	Tailwind CSS	Styling.	tailwindcss.com/docs: start with "Utility-first fundamentals" and "Responsive design"	3–5 days
11	Deploying with Vercel	Putting the site online.	vercel.com/docs: "Getting started" and "Deployments"	2 days
12	Testing with Playwright	Proving pages work before release.	playwright.dev/docs/intro	3–5 days

13	Web scraping, done responsibly	Collecting data without being blocked or breaking rules.	Read each site's Terms and robots.txt · "Web Scraping with Python" (Ryan Mitchell, O'Reilly; paid book) · requests + json docs	1 week

Times assume about 1–2 hours a day. Steps 1–5 and 6–10 can run in parallel if you prefer variety.

19.2 How to learn (the method matters more than the resource)

1. **Type, don't copy-paste.** Typing code yourself is how syntax sticks.
2. **Build tiny things right away.** After learning `fetch`, fetch one JSON file and print one field. After learning `useState`, build a counter.
3. **Break it on purpose.** Change a value, delete a line, see what error appears. Error messages become friends.
4. **Read real code.** This repository is small (about 1,400 lines). Read one file a day with this guide beside you.
5. **Explain it back.** If you can explain a file to a friend (or an AI) in plain words, you understand it.
6. **Keep a "today I learned" note.** One line a day makes a record of your progress.

19.3 How to use AI to learn (not just to get answers)

Good prompts to copy

- "Explain `useEffect` to a complete beginner, then give me a 10-minute exercise and don't show the answer until I try."
- "Here is `scrape_fotmob.py`. Walk me through it line by line in simple words. Stop after every 20 lines and quiz me with 2 questions."
- "I want to build X. Before writing code, ask me questions to define requirements, constraints and data sources, like a planning checklist."
- "Review my code like a strict teacher. Point out bugs and bad habits, but let me fix them myself."
- "Why did we choose a static JSON site instead of FastAPI + a database here? When would the other choice be better?"
- "I got this error: [paste]. Explain what it means and how to find the cause, but don't fix it for me yet."

Habits to avoid

Accepting code you can't explain; asking for a whole app in one prompt; skipping tests because "the AI said it works"; pasting secrets (tokens, passwords) into a chat. All four happened or nearly happened in this project; section 14 shows the fixes.

19.4 Practice projects that build up to this one

#	Mini project	Skills it trains
1	Python script that downloads a public JSON (e.g. a weather or public football API) and prints a summary.	requests, JSON, loops
2	Add a disk cache to project 1 so a second run makes no requests.	files, caching
3	Static web page (HTML/CSS/JS) that loads a local JSON file and shows a table.	fetch, DOM, tables

4	Rebuild project 3 in Next.js + TypeScript with a search box.	React state, components, types
5	Deploy project 4 to Vercel from GitHub.	git, deployment
6	A small version of this app: one league, current season only, players + goals.	the full pipeline-to-website flow

20 Roadmap: how to build a project like this from zero

Follow these steps in order for your next data website. Each step has a clear "done when..." so you know when to move on. This is exactly the order used in this project, cleaned up.

Step 1: Set up your tools

Day 0

Install VS Code, Git, Python 3, Node.js (LTS). Create GitHub and Vercel accounts and connect them.
Done when: `git --version`, `python --version` and `node --version` all print versions in your terminal.

Step 2: Write the requirements (one page)

Day 1

What will a visitor see and do? What data exactly (which fields, which seasons, which players)? What does "correct" mean?
Done when: you have a numbered list like section 2.1, and a sketch of each page on paper.

Step 3: Find and test the data source

Day 1–2

List candidate sources. Test each with one request (curl or Python). Check that it has every field you need, and read its terms. Pick a primary source and a second source for checking.
Done when: you have one saved real response from your primary source and know which keys hold your data.

Step 4: Choose the architecture with the planning questions

Day 2

Use the template in section 17.1: How often does the data change? Does anyone write data? How big is it? Read-only and updated daily or weekly → static data site (this project). Users write data or it changes every second → backend + database.
Done when: you can draw your version of Figure 2.

Step 5: Design the data files first

Day 3–4

Write the JSON shape the website will read (like section 9) and the matching TypeScript types. Decide the rules: unknown = null, how totals work.
Done when: you have one hand-written example JSON file for one item (one player).

Step 6: One example, end to end

Day 4–6

Write the smallest scraper for ONE item, the transform that produces your example JSON, and a bare web page that displays it.
Done when: one real item shows correctly on a local web page. (Our version: Haaland.)

Step 7: Make the scraper robust, then scale up

Week 2

Add disk cache, rate limit, retries, atomic writes and stages. Run on 10 items, then all items.
Done when: a full run completes and a second run makes zero requests.

Step 8: Verify the data

Week 2

Write invariant checks (totals add up, nothing negative, every file present) and an independent cross-check. Investigate every big difference.
Done when: 0 hard errors, a high agreement rate, and every remaining difference explained.

Step 9: Build the real website

Week 3

Next.js app: list page (load index, search, filter, sort) and detail page (load one file). Handle loading, error, empty and missing-image states.
Done when: `npm run lint` and `npm run build` pass, and you've clicked through 20 pages by hand.

Step 10: Test like a visitor

Week 3

A Playwright script that opens the home page and many detail pages, clicks the controls, and records errors. Also check that every data file loads.
Done when: the script reports 0 errors.

Step 11: Save, deploy a preview, test, go live

Week 4

Commit to a branch and push. `vercel deploy` (preview), test it, then `vercel promote` (production). Test the live URL again.
Done when: the public URL works on your phone and on a friend's computer.

Step 12: Keep it fresh and improve

Afterwards

Write a README with run commands. Automate the refresh (e.g. a weekly GitHub Actions job). Add features one at a time, each with steps 6 → 11.
Done when: someone else could run the project from your README alone.

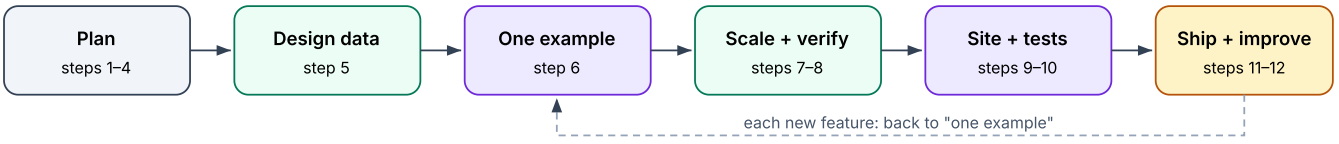


Figure 9. The roadmap at a glance.

The three rules to remember if you forget everything else

1. **Look at real data before you build.** Most bugs in this project came from assumptions about data shape.
2. **One example end to end, then scale.** Small loops are fast to fix.
3. **Never fake a number, never skip the test.** Show "–" for unknown; check against a second source; click through the real site before shipping.

End of guide. Everything described here is in the repository on branch `claude/data-pipeline-v2`.